Scalar SMU: Selective Recurrence Acceleration for Online Attention in a RISC-V Vector Cluster

Xiaoteng Wang

***Abstract*—Online attention merges tile-local results through a dependent scalar recurrence and a regular output-vector update. The Scalar Online-Merge Unit (SMU) accelerates this recurrence and reuses the existing Spatz/RISC-V Vector (RVV) datapath for the output update. Stage-aware mixed precision reduces hardware cost, and persistent context supports tile-level invocation. Across four native workloads, the MMIO-controlled Scalar SMU + RVV design achieves 8.524–17.784× recurrence speedup, 4.144–6.275× inclusive merge speedup, and 1.066–1.157× speedup in the measured Native Core kernel window over software recurrence with the same RVV update. Separate matched merge-microkernel ablations show that full offload requires 1.866–2.746× the selective design's merge cycles and 48.8% more standalone accelerator area. Mixed precision reduces standalone SMU mapped area by 27.6% in a separate arithmetic comparison. A paired mapping of the integrated cluster measures 1.606% additional standard-cell area, excluding SRAM macros. These results support recurrence specialization beside an existing vector processor and quantify its cycle benefit and mapped logic cost.**

***Index Terms*—RISC-V, online attention, selective acceleration, hardware/software co-design, vector processing, mixed precision.**

I. Introduction

Online attention avoids materializing the full attention matrix by merging tile-local results into a running state [1], [2]. This merge repeatedly couples a dependent scalar recurrence with a regular weighted update over the output dimension D. A programmable vector cluster already provides parallel arithmetic for the output update, but the recurrence combines maximum selection, exponential rescaling, accumulation, and normalization in a scalar dependency chain. Accelerating that chain while reusing vector resources provides a focused hardware opportunity within attention's score computation, normalization, and value aggregation [3].

We propose *selective recurrence acceleration*: a cluster-local Scalar Online-Merge Unit (SMU) generates the updated scalar state and two merge weights, and the existing Spatz/RISC-V Vector (RVV) datapath applies those weights to the output. The boundary follows the merge's dependency structure. Scalar arithmetic per query row is independent of D; the output update requires $O(D)$ independent multiply-add operations. A dedicated recurrence unit therefore complements the cluster's vector datapath without duplicating its output-update arithmetic.

Scalar SMU implements this boundary on a Spatz vector cluster [4] with stage-aware mixed precision and persistent tile-level control. Across four native workloads, MMIO-controlled Scalar SMU + RVV accelerates recurrence by 8.524–17.784× and the measured Native Core kernel window by 1.066–1.157× over software recurrence + the same RVV update. Standalone synthesis shows a 27.6% mapped-area reduction against Legacy Scalar, and a newer matched cluster mapping measures a 1.606% standard-cell area increment for SMU integration, excluding SRAM macros. A separate matched Full SMU ablation tests the partition directly: retaining RVV for the output update reduces both merge cycles and accelerator area relative to offloading that update as well.

Relation to prior work. Scalar SMU specializes the state transition between attention tiles. Softermax combines online normalization, base-two exponentiation, and reduced precision [5]. SoftEx provides dedicated Softmax/GELU execution beside a RISC-V cluster [6]. VEXP instead exposes approximate exponentiation through scalar and packed-SIMD instructions in the Snitch floating-point datapath [7]. Our unit operates on the running state and new tile statistics, producing updated state and two merge weights. The two weights expose a compact interface to the D-element RVV update. This interface places the specialization boundary *inside* the online merge, where dependent scalar arithmetic and regular vector arithmetic meet.

II. Online Merge and Architectural Insight

Let N denote the query-row count and, for the square workloads evaluated here, also the key/value sequence length. A key tile contains B positions, giving $T = N/B$ tiles; D is the output-vector dimension. For a query row, the state $(m, l, \mathbf{O})$ after processing scores s_i and value vectors $\mathbf{v}_i$ satisfies

$$m = \max_i s_i, \quad l = \sum_i e^{s_i - m}, \quad \mathbf{O} = l^{-1} \sum_i e^{s_i - m} \mathbf{v}_i. \tag{1}$$

A new tile provides its local state $(m_t, l_t, \mathbf{O}_t)$. The scalar merge is

$$\begin{aligned} m' &= \max(m, m_t), \\ l' &= l e^{m - m'} + l_t e^{m_t - m'}, \\ w_{\text{old}} &= \frac{l e^{m - m'}}{l'}, \qquad w_{\text{tile}} = \frac{l_t e^{m_t - m'}}{l'}. \end{aligned} \tag{2}$$

The vector output follows

$$O'[j] = w_{\text{old}} O[j] + w_{\text{tile}} O_t[j], \quad 0 \le j < D. \tag{3}$$

The first tile initializes the state directly. Each later tile performs Eqs. (2) and (3); the output remains normalized, so this representation needs no final normalization pass.

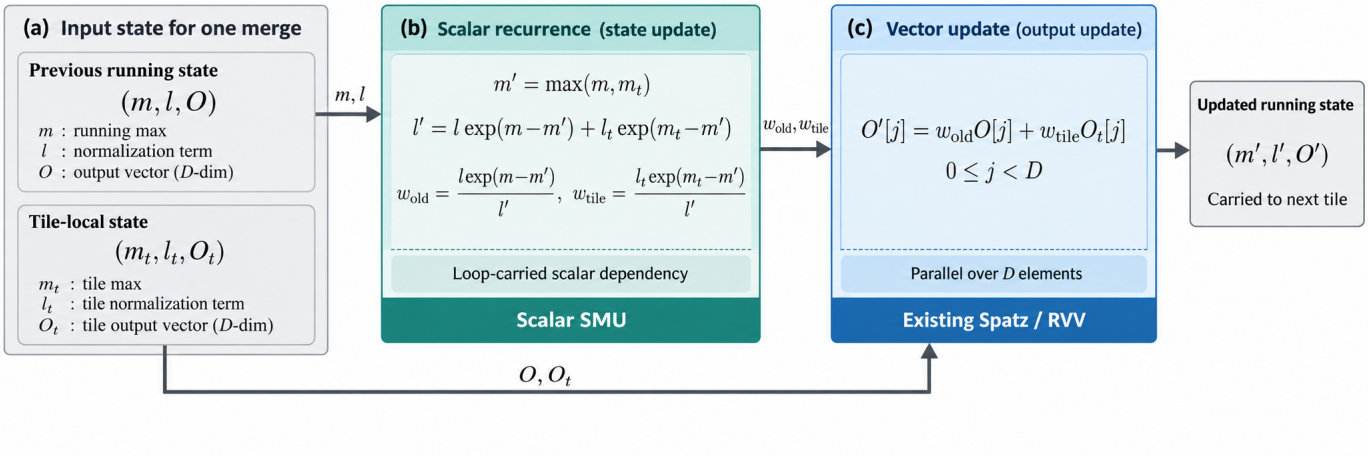


Fig. 1. Selective online-merge partition. The scalar recurrence updates (m, l) and generates two shared weights; the existing Spatz/RVV datapath applies them independently across the D output elements.

Figure 1 identifies the acceleration boundary. The state of a given query row carries across tiles, and the recurrence combines nonlinear arithmetic with dependent maximum, sum, and normalization operations. Its operation count per row is $O(1)$ in D. Different query rows have independent states, but the implemented SMU reuses one scalar datapath sequentially over all N rows. A tile-level command therefore still scales with N.

The output update has complementary structure: each element uses the same weights and has no dependency on other output elements. It maps directly to RVV multiply-add operations. Specializing it too would add a second execution path for work that the cluster already supports. The selective design instead spends dedicated arithmetic on the recurrence and retains vector execution, score generation, and tile-local state construction on the programmable path. The Full SMU ablation in Section IV-E tests this resource allocation by adding dedicated output-update execution.

The normalized-output formulation exposes two shared weights at every merge. FlashAttention-2 delays output normalization to reduce non-matrix-multiply work [8]; here, matched comparisons hold the normalized-output formulation fixed to evaluate the Scalar SMU/RVV partition.

III. Selective SMU Architecture

A. Shared-Memory Integration and Execution Model

The Snitch scalar core, Spatz/RVV, and SMU share tightly coupled data memory (TCDM), with 128 KiB in 16 banks in the default configuration. The SMU is an independent TCDM requester. Control requests traverse Snitch's accelerator request path and the cluster arbiter; responses return to the issuing core. Spatz retains its existing vector registers, floating-point units, and RVV instruction semantics [9], [4]. Figure 2 combines the integration and recurrence stages.

Two implementation-specific instructions separate setup from repeated execution. `OMCFG` writes the nine persistent context fields: base addresses for the A/B running-state arrays, tile-local maximum and normalizer, two weight arrays, and the query-row count N. A separate INIT operation commits this context and resets the A/B selector. `OMERGE` processes all N rows for one later score tile and returns completion status. Each configured scalar array stores one FP32 value per row. Output dimension D, tile scheduling, vector-buffer addresses, and strides remain software/RVV parameters.

Software initializes the first tile's running state in A and its output in a software-managed buffer. For each later tile, SMU reads the selected running state and tile statistics, writes the new (m', l') to the opposite A/B arrays, and stores both weights. The adapter flips the scalar-state selector only when the issuing core accepts a successful completion response; an error leaves it unchanged. Software then reads the weights through TCDM, invokes the RVV update, and advances its output-buffer pointers. The evaluated execution uses one controlling core and one active context. Scalar-state selection and vector-buffer ownership are separate responsibilities. Shared TCDM carries the updated state and weights between SMU and software/RVV.

B. Stage-Aware Mixed Precision

The fixed precision assignment follows the recurrence's numerical stages. FP32 is the memory-visible format. Maximum values and their differences use FP16 round-to-nearest-even (RNE). Exponential rescaling uses a 256-entry signed Q1.14 table with bin-center values and no interpolation:

$$\hat{s}(\Delta) = \begin{cases} 1, & \Delta \ge 0, \\ \mathrm{LUT}[\lfloor 32(\Delta+8) \rfloor], & -8 \le \Delta < 0, \\ 0, & \Delta < -8. \end{cases} \tag{4}$$

Here Δ is the difference after FP16 rounding. The winning maximum uses exact unity, and equal rounded maxima give unity on both paths. Normalizers are converted from FP32 to unsigned Q16.32, where the two scaled terms are accumulated. The denominator then crosses an FP16 RNE boundary. A

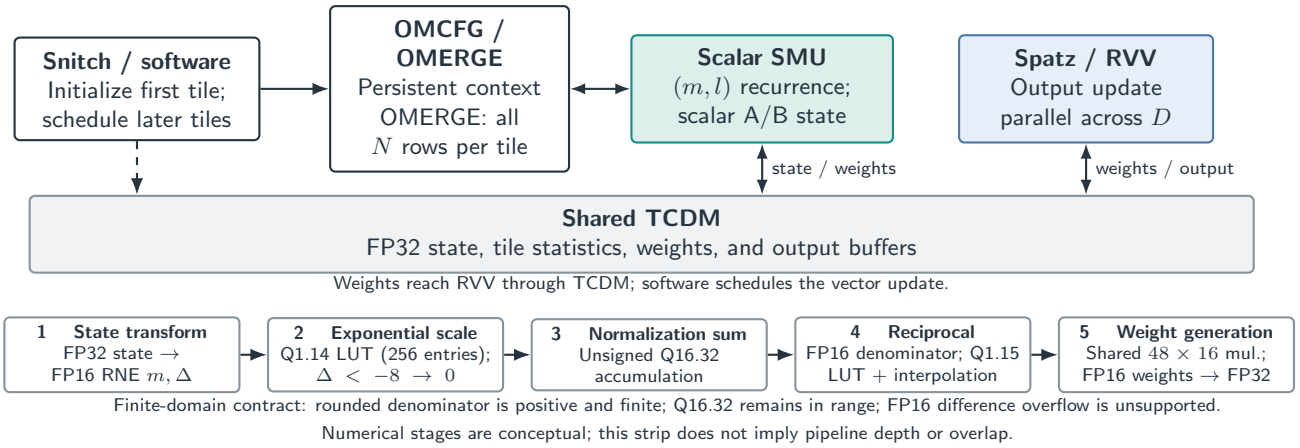


Fig. 2. System integration and recurrence implementation. Software configures persistent context and issues one OMERGE per later tile. SMU reads FP32 state and tile statistics and writes updated state and weights through shared TCDM; RVV reads the weights and updates the output vectors. The lower strip summarizes conceptual numerical stages, rather than pipeline depth.

Q1.15 reciprocal table with linear mantissa interpolation and exponent adjustment supplies its reciprocal.

Each scaled numerator also crosses an FP16 RNE boundary and returns to Q16.32 before weight generation. A shared 48×16 multiplier produces the two weights sequentially. Weights are rounded to FP16 and stored in FP32 for the RVV update; the maximum carries FP16 rounding, whereas the stored normalizer retains the Q16.32 accumulation through the FP32 interface. Only its reciprocal input crosses the FP16 denominator boundary. Each stage uses a fixed precision assignment. The arithmetic domain requires positive finite FP16-rounded denominators and in-range Q16.32 intermediates. Section IV-C measures numerical agreement and standalone mapped cost for this implementation.

IV. Evaluation

A. Setup and Measurement Scope

Cycle counts come from Verilator 5.034 simulation of the Spatz cluster with the default 128 KiB, 16-bank TCDM. Each matched comparison uses the same frozen cluster configuration and software toolchain. Four native workloads, N8/D32, N16/D64, N24/D64, and N16/D128, use $B = 4$ and seed one. They have $T - 1 = 1, 3, 5, 3$ later tiles, respectively; the first tile needs no merge.

The principal comparison is software recurrence + RVV against MMIO-controlled Mixed Scalar SMU + the same RVV output routine. The recurrence window covers state and weight generation; on the SMU path it also includes register programming, clear/start, polling, and engine execution. The inclusive merge window adds RVV output updates and merge-side orchestration. Native Core adds score-tile computation, local-state construction, first-tile handling, and final output copying. Inputs are already resident: allocation, input generation, Q/K/V loading, linear projections, and diagnostics are outside these windows. Speedup is baseline cycles divided by design cycles.

The principal comparison evaluates the combined Scalar SMU hardware and stage-precision assignment against the software recurrence's numerical formats. The instruction-interface implementation is evaluated separately from the MMIO arithmetic-offload results. Full SMU microkernels and standalone arithmetic synthesis each form separate matched families. We report cycle and area ratios within their respective implementation families. The integrated area experiment uses a separate frozen source closure and synthesis epoch; its area result does not establish the clock period of the cycle experiments.

B. Native Attention Performance

Figure 3 shows that recurrence specialization improves all four measured native kernels. At N8/D32, recurrence cycles fall from 12,368 to 1,451 and merge cycles from 13,204 to 2,318. Native Core drops from 85,262 to 73,706 cycles, a 1.157× speedup. At N16/D64, recurrence falls from 68,994 to 4,926 cycles and merge from 77,037 to 13,060; Native Core falls from 571,374 to 507,918 cycles, or 1.125×.

N24/D64 reaches 17.784× recurrence, 6.275× merge, and 1.140× Native Core speedup. N16/D128 reaches 13.737×, 4.144×, and 1.066×, respectively. These windows trace recurrence benefits through the merge to the measured attention kernel. Score-tile computation and local-state construction account for additional Native Core work, while RVV handles the dimension-dependent output update. At $N = 16$, merge speedups are 5.899× for $D = 64$ and 4.144× for $D = 128$, consistent with the larger output-update workload in the latter case.

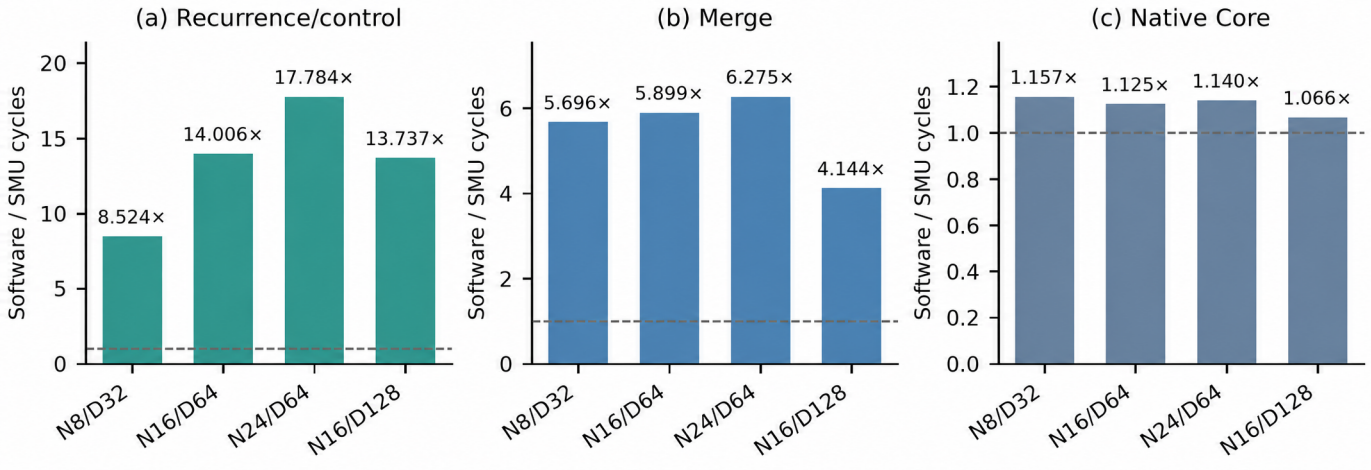


Fig. 3. Native workload speedups for MMIO-controlled Scalar SMU + RVV over software recurrence with the same RVV output routine. The recurrence/control, inclusive merge, and Native Core panels use separate vertical scales. All cases use $B = 4$ and seed one.

TABLE I
NATIVE OUTPUT AGREEMENT AND MAPPED-AREA COMPARISONS. ALL AREAS USE NANGATE45 LIBERTY UNITS. STANDALONE COLUMNS DENOTE SEPARATE HISTORICAL (H) AND CURRENT (C) SYNTHESIS EPOCHS; CLUSTER VALUES EXCLUDE SRAM MACROS.

(a) Difference from software recurrence + RVV		
Shape	MAE	Relative error
N8/D32	2.79×10^{-4}	0.49%
N16/D64	4.67×10^{-4}	0.71%
N24/D64	3.74×10^{-4}	0.61%
N16/D128	5.63×10^{-4}	0.52%
(b) Standalone arithmetic variants		
Design	Area (H)	Area (C)
Legacy Scalar	72,399	72,303
Mixed + Division	52,000	51,612
Mixed + Reciprocal	52,408	52,080
(c) Current integrated cluster pair		
Configuration	Cells	Area
SMU disabled	3,203,927	3,869,821
Scalar SMU + RVV	3,262,474	3,931,951

C. Hardware Cost and Numerical Agreement

Table I reports native output differences and standalone SMU synthesis. MAE and relative error are computed in float64 from stored FP32 outputs; relative error is $\sum |O_{\mathrm{SMU}} - O_{\mathrm{SW}}| / \sum |O_{\mathrm{SW}}|$. The reference is the matched software recurrence + RVV output. Relative error is 0.49%–0.71%, and no native-case output is nonfinite.

Matched Yosys/ABC synthesis with the Nangate45 typical library gives 52,408 area units for Mixed + Reciprocal, 27.6% below Legacy Scalar. Keeping division with mixed precision gives 52,000 units; reciprocal replacement adds 0.8% relative to that variant. The area reduction therefore comes primarily from the stage-aware precision assignment. Table I(b) separately reports a newer Yosys 0.70+221 reproduction with one mapping per role, alongside the historical Yosys 0.66+4 values. Both comparisons exclude RVV.

Separate host-model validation exhaustively covers all 31,743 positive finite FP16 denominators, with maximum reciprocal relative error 6.70×10^{-5}. A host scan covers random-score and single-head Q/K/V cases at lengths 128, 256, 512, and 1024 and dimensions 64 and 128: 16 combinations with $B = 32$ and base seed 20260823. Extended reciprocal-parameter scans report maximum relative error of 0.11% against the division version and 1.5% against an FP32 reference. These host-model checks use separate references and workloads from the four native RTL output-agreement cases. The extended optimized-versus-FP32 stress scan reaches a minimum cosine similarity of 0.999873853.

D. Integrated Standard-Cell Cost

The cluster pair in Table I(c) changes `EnableSmu` from zero to one in the same two-core `spatz_cluster_wrapper`, with VLEN-512 vector resources, DMA, interconnects, and peripherals. Both variants fix scalar-only mixed-precision reciprocal configuration; the disabled baseline retains common instruction decode and MMIO logic. Yosys 0.70+221 and ABC map the pair to Nangate45 typical cells at 1.10 V and 25°C without timing-driven constraints.

Standard-cell area increases from 3,869,820.668 to 3,931,950.820 units, an increment of 62,130.152 or 1.606%. This increment includes the engine, adapter, arbitration, routing, and other affected control logic; the engine alone occupies 59,846.542 units. Totals include latch-based vector register files and exclude 20 SRAM leaves: 16 TCDM banks and four instruction-cache arrays. Separate saved-netlist readback and Liberty-weighted cell histograms confirm the totals, with one valid mapping per variant.

OpenSTA analysis of these area netlists assumes an ideal 10 ns clock, without wire parasitics or SRAM timing arcs. Unbuffered high-fanout nets exceed Liberty characterization ranges and produce invalid delay extrapolations, preventing an operating-frequency conclusion. Separate standalone timing mappings report ABC delay proxies of 11.286, 14.141,

TABLE II
SEPARATE MATCHED MERGE-MICROKERNEL ABLATION. SELECTIVE INCLUDES THE SCALAR SMU RECURRENCE AND RVV OUTPUT UPDATE; FULL ADDITIONALLY OFFLOADS THE OUTPUT UPDATE. RATIOS ARE FULL/SELECTIVE CYCLES.

Shape	Selective	Full	Ratio
N12/D64	4,128	7,704	1.866×
N32/D128	12,866	34,728	2.699×
N40/D128	15,733	43,206	2.746×

and 16.495 ns for Legacy, Mixed + Division, and Mixed + Reciprocal. The reciprocal variant has the longest proxy; its area benefit does not establish a timing improvement.

E. Selective Partition and Dataflow Fit

To test the selective boundary, a separate matched merge-microkernel family compares Scalar SMU + RVV with Full SMU, which also offloads the output update. Table II reports three shapes using the same implementation and timing window within each pair. Full SMU needs 1.866–2.746× the selective design's merge cycles. Its standalone area is 114,713 units versus 77,103 for Scalar SMU, an increase of 48.8%. Both area values belong to the matched ablation implementation; the mixed-precision arithmetic variants appear in Table I.

Retaining RVV for the regular update reduces merge cycles across all three ablation shapes and avoids the additional Full SMU area. This matched merge-microkernel comparison supports the selective boundary; the native comparison measures its benefit against software recurrence at kernel scope. Together, they support a recurrence datapath that supplies shared weights to existing vector execution.

A negative control evaluates a separate explicit-probability-matrix dataflow. Replacing row-wise software Softmax while holding the surrounding quantized-attention path fixed increases its Softmax cycles by 18.477% at N8/D32 and reduces them by 0.126% at N16/D64. The five-stage attention totals increase by 0.296% and 0.015%, respectively. Initialization, launch/polling, RVV probability updates, and buffer orchestration remain in that measured path. This result supports targeting native online recurrence rather than treating SMU as a general row-wise Softmax replacement.

V. DISCUSSION

The native kernels and full-offload ablation support allocating dedicated arithmetic to the scalar transition while RVV retains the parallel output update. Smaller Native Core gains reflect the remaining score computation and local-state work. The explicit-probability negative control shows that invocation and buffer costs can outweigh benefits in a different dataflow. This partition complements online normalization and approximate Softmax arithmetic [1], [5], [7] by defining which state and outputs the dedicated unit owns. FlashAttention-2's deferred normalization changes that boundary's workload [8]; our comparisons hold the normalized-output formulation fixed.

The four small resident-input workloads use one seed and compare the combined hardware and numerical design. They do not isolate approximation's contribution. The integrated area pair uses a newer synthesis epoch than the cycle experiments. Physical timing and power remain open: the results quantify cycles and mapped standard-cell cost separately, without establishing wall-clock or energy savings.

VI. CONCLUSION

Scalar SMU places dedicated arithmetic at the state-dependent recurrence inside online attention and reuses Spatz/RVV for the regular D-element output update. Stage-aware mixed precision reduces recurrence hardware cost, and persistent tile-level control exposes the partition to software. Across four native workloads, the MMIO-controlled design reaches 1.066–1.157× measured Native Core speedup, with 8.524–17.784× recurrence and 4.144–6.275× merge speedups. Separate matched full-offload microkernels require more cycles and area, while standalone mixed-precision synthesis reduces Scalar SMU area by 27.6% against Legacy Scalar. A separate current cluster mapping measures a 1.606% standard-cell area increment, excluding SRAM macros. The results support placing dedicated hardware at the recurrence boundary when regular output updates can use an existing vector processor.

REFERENCES

[1] M. Milakov and N. Gimelshein, "Online normalizer calculation for Softmax," arXiv preprint arXiv:1805.02867, 2018, doi: 10.48550/arXiv.1805.02867.

[2] T. Dao, D. Y. Fu, S. Ermon, A. Rudra, and C. Ré, "FlashAttention: Fast and memory-efficient exact attention with IO-awareness," in *Advances in Neural Information Processing Systems*, vol. 35, 2022, pp. 16 344–16 359, doi: 10.52202/068431-1189.

[3] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, "Attention is all you need," in *Advances in Neural Information Processing Systems*, vol. 30, 2017, pp. 5998–6008.

[4] M. Perotti, S. Riedel, M. Cavalcante, and L. Benini, "Spatz: Clustering compact RISC-V-based vector units to maximize computing efficiency," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 44, no. 7, pp. 2488–2502, 2025, doi: 10.1109/TCAD.2025.3528349.

[5] J. R. Stevens, R. Venkatesan, S. Dai, B. Khailany, and A. Raghunathan, "Softermax: Hardware/software co-design of an efficient Softmax for transformers," in *2021 58th ACM/IEEE Design Automation Conference (DAC)*, 2021, pp. 469–474, doi: 10.1109/DAC18074.2021.9586134. [Online]. Available: https://doi.org/10.1109/DAC18074.2021.9586134

[6] A. Belano, Y. Tortorella, A. Garofalo, L. Benini, D. Rossi, and F. Conti, "A flexible template for edge generative AI with high-accuracy accelerated Softmax and GELU," *IEEE J. Emerg. Sel. Topics Circuits Syst.*, vol. 15, no. 2, pp. 200–216, 2025, doi: 10.1109/JETCAS.2025.3562734.

[7] R. Wang, G. Islamoglu, A. Belano, V. Potocnik, F. Conti, A. Garofalo, and L. Benini, "VEXP: A low-cost RISC-V ISA extension for accelerated Softmax computation in transformers," in *Proc. IEEE Symp. Computer Arithmetic (ARITH)*, 2025, pp. 37–44, doi: 10.1109/ARITH64983.2025.00016.

[8] T. Dao, "FlashAttention-2: Faster attention with better parallelism and work partitioning," in *International Conference on Learning Representations (ICLR)*, 2024.

[9] F. Zaruba, F. Schuiki, T. Hoefler, and L. Benini, "Snitch: A tiny pseudo dual-issue processor for area and energy efficient execution of floating-point intensive workloads," *IEEE Transactions on Computers*, vol. 70, no. 11, pp. 1845–1860, 2021, doi: 10.1109/TC.2020.3027900. [Online]. Available: https://doi.org/10.1109/TC.2020.3027900